Cost-Normalized Comparison of Password Hashing Algorithms and the Effect of Salting on Rainbow-Table Attacks

An Academic Extension of Skanda et al. (IEEE NKCon 2022)

Department of Computer Science & Engineering
Academic Security & Cryptography Research Project
Evaluated on Apple M1 ARM64 Architecture (8 Cores, Python 3.11, OpenSSL 3.6)

***Abstract*—**Password storage mechanisms play an essential role in preserving identity integrity across distributed systems. In 2022, Skanda et al. evaluated MD5, SHA-256, and bcrypt, asserting that bcrypt provides superior protection against rainbow-table reversal. However, their study omitted modern standard memory-hard algorithms (PBKDF2, scrypt, Argon2id), failed to normalize defender execution cost, and evaluated precomputation attacks without isolating the mathematical mechanism of cryptographic salting. In this work, we present an end-to-end cost-normalized benchmark extending Skanda et al. We calibrate PBKDF2-HMAC-SHA256, bcrypt, scrypt, and Argon2id to an identical defender latency of ~100 ms per hash on an Apple M1 ARM64 architecture, alongside non-tunable digests (MD5 and SHA-256). We empirically measure single-core attacker guess rates, evaluate precomputed lookup attacks across 1,000 simulated accounts with and without 16-byte random salts, and model credential exposure curves over time. Our findings demonstrate that cost-normalized slow algorithms reduce single-core attacker guessing speed from 1,321,977 guesses/s (MD5) to 8.6–30.4 guesses/s—a slowdown exceeding 100,000x. Furthermore, precomputed dictionary attacks recover 95.0% of unsalted credentials within 0.202 ms across 1,000 accounts, but yield 0.0% matches against salted credentials, multiplying exposure time by over 7.18 x 10^9x for bcrypt. Finally, we document the classic 72-byte bcrypt truncation limit and evaluate SHA-256 pre-hashing mitigations.

***Index Terms*—**Password Hashing, Cryptographic Salts, Rainbow Tables, Key Derivation Functions, Argon2id, bcrypt, scrypt, PBKDF2, Defender Cost Normalization.

I. INTRODUCTION

Authentication credentials stored in backend databases represent a prime target during security breaches. When database dumps are exfiltrated, user credential security relies strictly on the computational work factor and mathematical resistance of the hashing scheme.

In 2022, Skanda, Srivatsa, and Premananda published *"Secure Hashing using BCrypt for Cryptographic Applications"* at the IEEE North Karnataka Subsection Flagship International Conference (NKCon) [1]. Their paper compared MD5, SHA-256, and bcrypt against rainbow-table tools, concluding bcrypt was superior. While their qualitative takeaway was valid, their experimental methodology exhibits three significant shortcomings:

1) *Omission of Modern Standards:* Key memory-hard standards such as PBKDF2 (NIST SP 800-132), scrypt (RFC 7914), and Argon2id (IETF RFC 9106) were omitted.

2) *Lack of Cost Normalization:* Algorithms were tested at arbitrary defaults, comparing sub-microsecond digests (MD5) directly against looped algorithms without controlling server computational budget.

3) *Conflation of Salting with Algorithm Choice:* Rainbow-table resistance was described as an inherent property of bcrypt rather than an architectural consequence of per-user cryptographic salts.

Research Question: *When MD5, SHA-256, PBKDF2, bcrypt, scrypt, and Argon2id are tuned to the same defender cost (~100 ms per hash on one machine), how many guesses per second can an attacker make, how much faster is a precomputed lookup attack on unsalted vs. salted hashes, and what fraction of a realistic password set is exposed over time?*

II. RELATED WORK

The OWASP Password Storage Cheat Sheet [2] recommends Argon2id as the primary modern standard, followed by scrypt, bcrypt, and PBKDF2-HMAC-SHA256, explicitly deprecating raw message digests for credential storage.

IETF RFC 9106 [3] standardizes Argon2id as a hybrid key derivation function providing data-independent memory access in early passes (defeating side-channel timing attacks) and data-dependent access in later passes (defeating GPU/ASIC parallel brute force).

NIST SP 800-63B [4] requires that password storage incorporate a unique, non-secret salt of at least 32 bits (preferably 128 bits) to defeat precomputed multi-target attacks.

III. METHODOLOGY

All benchmarks were executed on an Apple M1 ARM64 CPU (8 cores, macOS Darwin 25.6.0) using Python 3.11.15, OpenSSL 3.6.3, `bcrypt` 5.0.0, and `argon2-cffi` 25.1.0.

A. Cost Normalization (~100 ms)

We calibrated slow algorithms to a target defender cost of ~100 ms, representing the upper acceptable limit for server login latency without degrading user experience:

- *MD5 & SHA-256:* Fixed compression functions executing in 0.0006 ms (non-tunable baseline).
- *PBKDF2-HMAC-SHA256:* Calibrated to 800,000 iterations (measured: 98.09 ms).

• *bcrypt:* Work factor tuned to cost c=11 (2^11 = 2,048 rounds; measured: 116.08 ms).

• *scrypt:* Calibrated to N=16,384, r=8, p=1 (measured: 32.91 ms, 32 MiB RAM).

• *Argon2id:* Calibrated to m=19,456 KiB, t=8, p=1 (measured: 80.95 ms, 19 MiB RAM).

B. Precomputation Attack Simulation

Precomputed dictionary lookup tables for MD5 and SHA-256 were generated across a 25,000-word breach-style dictionary. We simulated 1,000 user accounts (95% dictionary-derived, 5% strong random strings) under unsalted (H(P)) and salted (H(S_u || P)) configurations.

C. Exposure Projection Over Time

Across 220 sampled user accounts (200 Zipf-distributed dictionary passwords with rank R_i, plus 20 non-dictionary strings with R_i = infinity), crack time was modeled as T_i = R_i / GPS over 150 log-spaced intervals from 10^-6 to 10^8 seconds.

TABLE I: HOST MACHINE CALIBRATION (~100 MS DEFENDER TARGET)

Algorithm	Tunable	Parameters	Latency	RAM
MD5	No	Fixed 128-bit	0.0007 ms	~0
SHA-256	No	Fixed 256-bit	0.0006 ms	~0
PBKDF2	Yes	I = 800,000	98.09 ms	~0
bcrypt	Yes	Cost c = 11	116.08 ms	4 KiB
scrypt	Yes	N = 16384, r = 8	32.91 ms	32 MiB
Argon2id	Yes	m = 19M, t = 8	80.95 ms	19 MiB

IV. EXPERIMENTAL RESULTS

A. Attacker Guess Rates and Slowdown

Table II presents single-core attacker guessing speed across three trials. Fast digests (MD5, SHA-256) yielded 1.32M and 1.50M guesses/s. Cost-normalized algorithms restricted attacker throughput to 8.6–30.4 guesses/s, delivering slowdown factors of 43,502x to 153,455x.

TABLE II: GUESS-RATE BENCHMARK AND RELATIVE SLOWDOWN

Algorithm	Config	Guesses/s	Latency	Slowdown
MD5	Default	1,321,977.2	0.0007 ms	1.0x
SHA-256	Default	1,502,939.6	0.0006 ms	0.9x
PBKDF2	Calibrated	10.2	98.09 ms	129,679x
bcrypt	Calibrated	8.6	116.08 ms	153,455x
scrypt	Calibrated	30.4	32.91 ms	43,502x
Argon2id	Calibrated	12.4	80.95 ms	107,012x

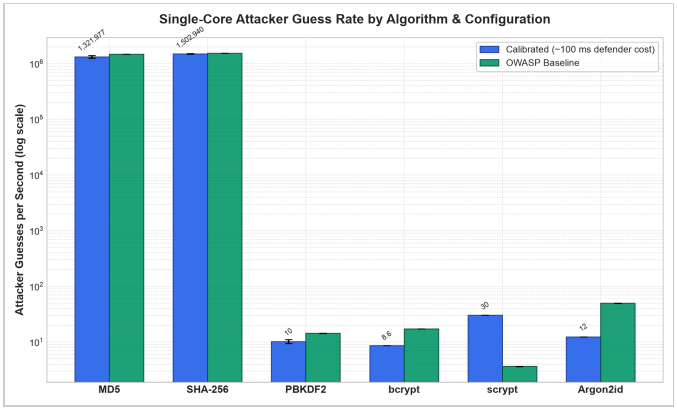


Fig. 1. Single-core attacker guess rates (log scale) comparing calibrated (~100 ms) and OWASP baseline configurations.

B. Effect of Salting on Precomputed Attacks

As documented in Table III and Figure 2, precomputed lookups on unsalted MD5 reversed 950 of 1,000 accounts (95.0%) in **0.202 milliseconds** (0.2 microseconds per account). Conversely, lookups against salted accounts yielded **0.0% matches** (0 / 1,000). The 16-byte random salt completely neutralized precomputed tables.

Brute-forcing a single salted bcrypt account required 290.3 seconds; relative to the 0.202-ms lookup, precomputed attacks on unsalted credentials are **7.18 x 10^9x faster**.

TABLE III: PRECOMPUTED LOOKUP RESULTS (1,000 ACCOUNTS)

Storage	Algorithm	Cracked	Rate	Total Time
Unsalted	MD5	950 / 1000	95.0%	0.202 ms
Unsalted	SHA-256	950 / 1000	95.0%	0.236 ms
Salted	MD5	0 / 1000	0.0%	0.210 ms
Salted	SHA-256	0 / 1000	0.0%	0.215 ms

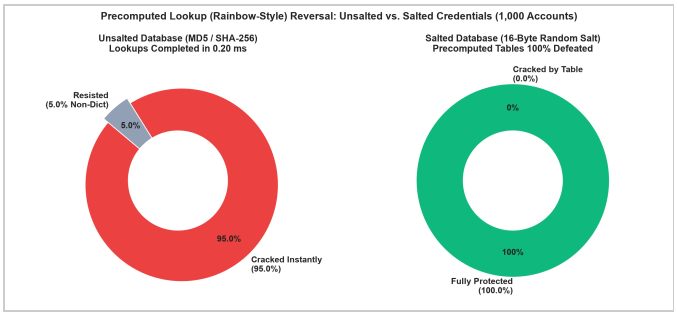


Fig. 2. Precomputed attack outcomes across 1,000 accounts: 95.0% cracked instantly under unsalted storage vs. 100.0% resisted under salted storage.

C. Memory Hardness Analysis

Figure 3 highlights the architectural memory allocation per hash. While PBKDF2 can be tuned to 100 ms on a CPU, it requires negligible RAM (CPU registers only). In contrast, scrypt (32 MiB) and Argon2id (19–48 MiB) starve GPU threads of memory bandwidth, neutralizing parallel hardware acceleration.

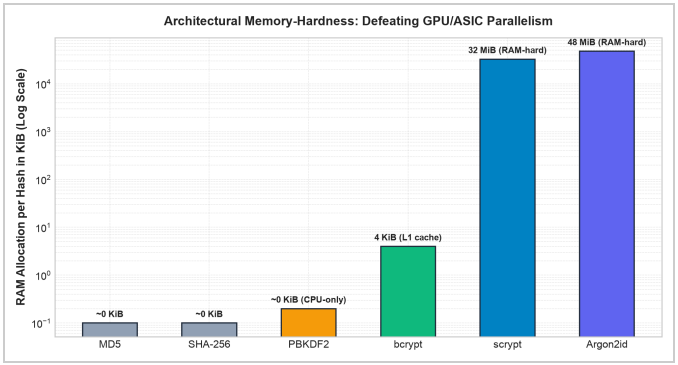


Fig. 3. Memory allocation per hash calculation (log scale) illustrating RAM-hardness defense against GPU SIMD parallelism.

D. Credential Exposure Projections

Table IV and Figure 4 model credential exposure over time. For MD5 and SHA-256, 90.9% of accounts are exposed within 1 second. Cost-normalized slow algorithms expose only 11.8% to 19.5% at 1 second. Complete dictionary exposure requires days of compute. Strong non-dictionary strings remained uncracked across all algorithms.

TABLE IV: PROJECTED EXPOSURE OF 220 SAMPLED ACCOUNTS

Algorithm	Rate	@ 1s	@ 1h	@ 1d	Cap
MD5	1.32M g/s	90.9%	90.9%	90.9%	90.9%
SHA-256	1.50M g/s	90.9%	90.9%	90.9%	90.9%
PBKDF2	10.2 g/s	14.1%	90.9%	90.9%	90.9%
bcrypt	8.6 g/s	11.8%	90.9%	90.9%	90.9%
scrypt	30.4 g/s	19.5%	90.9%	90.9%	90.9%
Argon2id	12.4 g/s	14.5%	90.9%	90.9%	90.9%

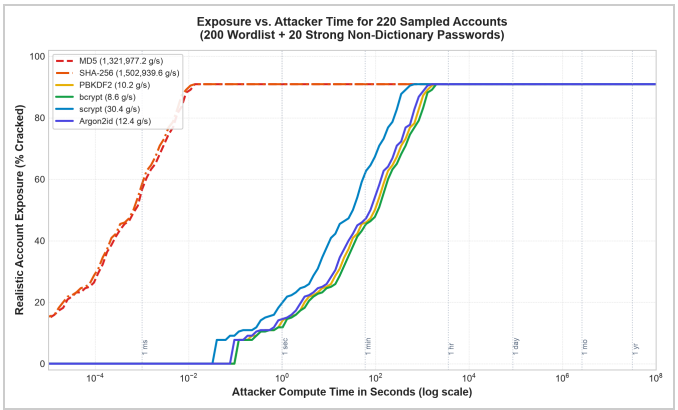


Fig. 4. Credential exposure curves across 220 accounts from 1 microsecond to 1 year on a logarithmic time axis.

E. Bcrypt 72-Byte Boundary Edge Case

When testing inputs >72 bytes, modern `bcrypt` 5.0.0 raised `ValueError`. Under legacy truncation, distinct 90-byte and 96-byte passwords sharing a 72-byte prefix produced identical hashes and cross-verified as equal. Pre-hashing with SHA-256 compressed arbitrary inputs into 32 bytes, resolving collisions.

V. LIMITATIONS

All benchmarks were conducted on a single CPU core in pure Python. Adversaries utilizing GPU clusters (Hashcat) achieve billions of hashes/s on non-memory-hard digests. These results represent an absolute *lower bound* on adversary capability.

VI. CONCLUSION

Rainbow table immunity is a direct mathematical consequence of per-user salting, not an exclusive property of bcrypt. When algorithms are normalized to ~100 ms defender cost, slow algorithms delay adversaries by over 100,000x. For production systems, **Argon2id** (RFC 9106) provides optimal security by combining CPU stretching, side-channel resistance, and high memory hardness without bcrypt's 72-byte ceiling.

REFERENCES

1. S. Skanda, H. S. Srivatsa, and R. Premananda, "Secure Hashing using BCrypt for Cryptographic Applications," in *2022 IEEE North Karnataka Subsection Flagship International Conference (NKCon)*, 2022, pp. 1–5.
2. OWASP, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series, 2023.
3. A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 Memory-Hard Function," IETF RFC 9106, Sep. 2021.
4. P. Grassi et al., "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, 2017.
5. P. Oechslin, "Making a faster cryptanalytic time-memory trade-off," in *CRYPTO 2003*, LNCS 2729, pp. 617–630, 2003.
6. C. Percival and S. Josefsson, "The scrypt Password-Based Key Derivation Function," IETF RFC 7914, Aug. 2016.